

Qué orden conoce un método

Condiciones de orden de Runge–Kutta certificadas sobre árboles enraizados, comprobadas por máquina en Lean 4

Carles Marín*

Investigador independiente

karlesmarin@gmail.com

Junio de 2026

Resumen

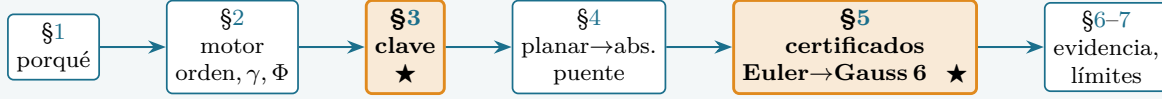
Un método de Runge–Kutta es una tabla de números (A, b) ; su *orden* es el mayor p para el que su error de un paso es $O(h^{p+1})$. El teorema de Butcher convierte esa pregunta analítica en combinatoria pura: el método tiene orden p exactamente cuando, para todo árbol enraizado t con a lo sumo p nodos, un *peso elemental* $\Phi(t)(A, b)$ es igual a $1/\gamma(t)$, donde γ es la densidad del árbol. Estas *condiciones de orden* se calculan a diario—con NodePy, con `BSeries.jl`, con paquetes de Mathematica y Maple—pero siempre como *scripts de confianza* no verificados. Este artículo formaliza las condiciones de orden *en sí* en Lean 4, todas sin `sorry` y con el conjunto mínimo de axiomas `{propext, Classical.choice, Quot.sound}`. Construimos un pequeño motor computable $(\text{orden}(t), \gamma(t), \Phi)$, probamos una *clave* (keystone) que reduce la familia infinita de condiciones de orden a una comprobación finita y decidible sobre un catálogo, y la descargamos para los métodos clásicos: Euler (orden 1), el trapecio explícito (orden 2), el RK4 clásico (orden 4), Dormand–Prince `DOPRI5` (orden 5, el método por defecto en MATLAB `ode45` y SciPy `RK45`), y el método de Gauss–Legendre de 3 etapas —totalmente implícito, con coeficientes en $\mathbb{Q}(\sqrt{15})$ —cuyas condiciones de orden 6 quedan a la vez confirmadas de forma independiente en Sage y *comprobadas por máquina en Lean hasta el orden 6 completo*—mediante una reformulación con aritmética entera que vuelve el certificado decidible por el núcleo; véase la Sección 6. Los árboles se codifican de forma *planar*, como el álgebra de Hopf no conmutativa de Connes–Kreimer–Foissy; un lema corto de simetría— Φ es invariante al permutar los hijos de un nodo—es la sombra formal del morfismo de abelianización de Foissy hacia el álgebra de Butcher abstracta, y muestra que certificar todos los árboles planares *subsume* las condiciones de orden habituales (no planares). Hasta donde sabemos, tras una búsqueda dirigida en las bibliotecas de Lean/Mathlib, Isabelle/HOL y Coq/Rocq, las condiciones de orden de Runge–Kutta no se habían formalizado antes en ningún asistente de pruebas; el único trabajo previo de métodos formales sobre RK (Boldo *et al.*, Coq) certifica el redondeo en coma flotante de Euler y RK2 sobre sistemas lineales, una capa ortogonal. Somos exactos con la frontera: certificamos las condiciones de orden *algebraicas* $\gamma(t)\Phi(t) = 1$, no el teorema analítico de que equivalen a un orden de convergencia; la matemática es clásica en todo momento (Butcher 1963; Foissy 2002); y “primero en un asistente de pruebas” es el resultado de una búsqueda dirigida, no un teorema. La contribución es la formalización y un comprobador de condiciones de orden certificado y reutilizable.

*Se usó a Claude (Anthropic) como asistente; cada enunciado fue verificado de forma independiente por el núcleo (kernel) de Lean, y toda la matemática y las afirmaciones son responsabilidad del autor.

Mapa del lector

Si solo lees dos cosas, lee el **Teorema 1** (la clave) y la **Tabla 4** (un certificado resuelto). El primero convierte “tiene orden p ”—un enunciado sobre infinitos árboles enraizados— en una única comprobación finita y decidible; la segunda muestra esa comprobación en acción, con RK4 pasando toda condición de orden ≤ 4 y fallando en el orden 5.

La ruta (las cajas con estrella son las dos que leer primero):



Las fórmulas que sostienen el argumento van enmarcadas; los límites honestos (§7) se enuncian con la misma claridad que los resultados. *Reproducibilidad:* `lean/ButcherOrder.lean` es autocontenido; `lake env lean` lo recompila y `#print axioms` reporta el conjunto mínimo `{propext, Classical.choice, Quot.sound}` en cada teorema cabecera; los cruces numéricos viven en `app/bseries.py` y `validate_order_conditions.sage`. El artefacto (fuente Lean + cruces) está archivado en Zenodo: DOI 10.5281/zenodo.20787666.

1. Por qué esto, por qué ahora

¿Cómo *sabes* que un integrador numérico tiene el orden que afirmas? En la práctica te fías de un script. Las condiciones de orden son una pieza de combinatoria hermosa y finita—una ecuación por árbol enraizado— pero el puente de “mi código imprimió `True`” a “esta ecuación se cumple sobre $\mathbb{Q}$ ” queda sin auditar. Este artículo aporta una implementación en Lean reutilizable y comprobada por máquina que aborda ese hueco para las condiciones mismas: un certificado, recompilable por cualquiera, de que un tableau dado satisface toda condición de orden hasta el orden p .

El diferencial. Calcular condiciones de orden a partir de árboles enraizados es terreno trillado: NodePy [9], BSeries.jl/RootedTrees.jl [10], el paquete de Mathematica de Sofroniou [11], herramientas de Maple y MATLAB—todas excelentes, todas código simbólico/numérico *no verificado*. Del lado de los asistentes de pruebas, una búsqueda dirigida (Sección 7) no halló formalización alguna de la teoría de orden de árboles enraizados en Lean, Coq o Isabelle; el único resultado RK comprobado por máquina, Boldo–Faissolle–*et al.* [8], certifica el error de *redondeo* de Euler/RK2 sobre sistemas lineales—una capa enteramente distinta. Así que las condiciones de orden mismas—el corazón combinatorio— parecen no haber sido formalizadas (véase la búsqueda en la Sección 7). La matemática es clásica (Butcher [1]; Hairer–Lubich–Wanner [2]; Chartier–Hairer–Vilmart [3]); la novedad es la formalización.

Lugar en una serie. Este artículo es el *applied payoff* prometido en la formalización compañera del álgebra de Hopf de árboles enraizados [17], que construye el álgebra de Connes–Kreimer/Butcher en Lean pero se detiene, deliberadamente, antes de sus usos. Esa álgebra *es* el álgebra de estas condiciones de orden: el peso elemental $\Phi(t)(A, b)$ es un *carácter* suyo, el flujo exacto es el carácter $t \mapsto 1/\gamma(t)$, y un método tiene orden p exactamente cuando ambos caracteres coinciden en todos los árboles de orden $\leq p$. La Sección 9 precisa este diccionario.

2. El motor: orden, densidad, peso elemental

¿Cuál es el objeto computable más pequeño que decide una condición de orden? Un árbol enraizado, codificado sin cociente como `inductive RTree | node : List RTree`. Tres recursiones:

$$\text{orden}(\text{node } F) = 1 + \sum_{c \in F} \text{orden}(c), \quad \gamma(\text{node } F) = \text{orden}(\text{node } F) \cdot \prod_{c \in F} \gamma(c).$$

Para un tableau (A, b) sobre un anillo conmutativo K , el vector de pesos internos y el peso elemental son

$$g(t)_i = \prod_{c \in F} (A g(c))_i \quad (t = \text{node } F), \quad \Phi(t) = b^\top g(t).$$

La condición de orden en t se enuncia de forma multiplicativa, de modo que el motor solo necesita un anillo conmutativo (sin división): $\gamma(t) \Phi(t) = 1$, equivalente a $\Phi(t) = 1/\gamma(t)$ ya que $\gamma(t) \geq 1$. Todo se basa en `List` y es computable, así que el núcleo lo evalúa; los valores de `#eval` se cruzaron con la verdad de referencia de Sage/Python (`app/bseries.py`, `validate_order_conditions.sage`). El mismo motor trata sin cambios los métodos *implícitos*: las condiciones de orden son puramente algebraicas en (A, b) y nunca resuelven el sistema de etapas implícito.

árbol t	$ t $	$\gamma(t)$	condición de orden	$\Phi(t) = 1/\gamma(t)$
	1	1	$\sum_i b_i = 1$	
	2	2	$\sum_i b_i c_i = \frac{1}{2}$	
	3	3	$\sum_i b_i c_i^2 = \frac{1}{3}$	
	3	6	$\sum_{i,j} b_i a_{ij} c_j = \frac{1}{6}$	
	4	4	$\sum_i b_i c_i^3 = \frac{1}{4}$	
	4	8	$\sum_{i,j} b_i c_i a_{ij} c_j = \frac{1}{8}$	
	4	12	$\sum_{i,j} b_i a_{ij} c_j^2 = \frac{1}{12}$	
	4	24	$\sum_{i,j,k} b_i a_{ij} a_{jk} c_k = \frac{1}{24}$	

Tabla 1: Las condiciones de orden hasta el orden 4, una por árbol enraizado, con $c_i = \sum_j a_{ij}$. El peso elemental $\Phi(t)$ es exactamente la suma de la izquierda; la densidad $\gamma(t)$ da el objetivo. Estos son los ocho árboles abstractos; el motor comprueba los nueve representantes planares, colapsados por el Lema 1.

La tabla anterior se detiene en el orden 4 solo por la página; el patrón no. “Orden p ” exige una ecuación así para *todo* árbol enraizado con a lo sumo p nodos, y hay infinitos árboles enraizados. Un certificado no puede comprobar infinitas ecuaciones—así que el primer obstáculo real no es la aritmética sino la cuantificación. La eliminamos a continuación.

3. La clave: infinitas condiciones, una comprobación finita

“Orden p ” cuantifica sobre todos los árboles enraizados—¿cómo es eso decidible?

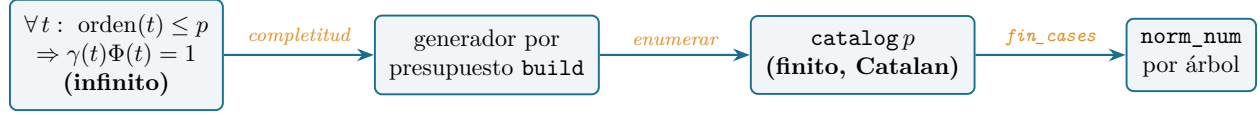


Figura 1: La clave (Teorema 1): la familia infinita de condiciones de orden se reduce, mediante un generador por presupuesto de orden demostrablemente completo, a un catálogo finito que `fin_cases` parte y `norm_num` cierra árbol por árbol.

Definimos

$$\text{satisfiesOrderConditions } (A, b) \, p := \forall t, \text{orden}(t) \leq p \rightarrow \gamma(t) \Phi(t) = 1.$$

Construimos un generador *por presupuesto de orden* `build : Nat -> List RTree x List Forest` (estructural sobre el presupuesto; su lista de árboles tiene longitud Catalan acumulada 1, 2, 4, 9, 23, 65, ...), probamos su completitud

$$\text{orden}(t) \leq n \Rightarrow t \in (\text{build } n), 1$$

por inducción estructural mutua con medida (presupuesto, tamaño del árbol), y obtenemos la

Teorema 1 (clave). $\text{satisfiesOrderConditions } (A, b) \, p \iff \forall t \in \text{catalogo } p, \gamma(t) \Phi(t) = 1$, donde `catalogo p` es la lista finita de todos los árboles enraizados con $\text{orden}(t) \leq p$.

La solidez ($t \in \text{catalogo } p \Rightarrow \text{orden}(t) \leq p$) es el filtro; la completitud es el lema del generador. El lado derecho es una conjunción finita, descargada árbol por árbol.

orden n	1	2	3	4	5	6
árboles planares de orden n (Catalan C_{n-1})	1	1	2	5	14	42
<code> catalogo n </code> (planares acumulados)	1	2	4	9	23	65
árboles abstractos de orden n (A000081)	1	1	2	4	9	20
abstractos acumulados	1	2	4	8	17	37

Tabla 2: El catálogo es sobre árboles *planares* (ordenados), contados por los números de Catalan; los árboles de Butcher abstractos son menos (A000081). El Lema 1 colapsa planar a abstracto, de modo que la comprobación del catálogo planar subsume las condiciones de orden abstractas. Tamaños verificados con `#eval`.

4. Árboles planares y el puente a las condiciones abstractas

Nuestros árboles tienen hijos **ordenados**; los de Butcher, *multiconjuntos*—¿cambia eso las condiciones? El `RTree` basado en `List` es el álgebra de Connes–Kreimer *planar* (no conmutativa) estudiada por Foissy [5, 6] (y, para árboles binarios planares, Loday–Ronco [12]); los árboles enraizados planares se cuentan por los números de Catalan [16] (1, 1, 2, 5, 14, ...), los abstractos por OEIS A000081 [15] (1, 1, 2, 4, 9, ...). El álgebra de Butcher / Connes–Kreimer conmutativa es

la *abelianización* de la planar—el morfismo de Hopf graduado y sobreyectivo de Foissy (olvidar el orden planar), con núcleo el ideal de conmutadores [5, Prop. 81]. Formalizamos la sombra pertinente de ese morfismo:

Lema 1 (simetrización). *Si F y G son listas de hijos relacionadas por una permutación, entonces $\gamma(\text{node } F)\Phi(\text{node } F) = 1 \iff \gamma(\text{node } G)\Phi(\text{node } G) = 1$.*

El motor de la prueba es que el producto puntual $g(t)_i = \prod_c (Ag(c))_i$ se construye a partir de una operación conmutativa-asociativa, de modo que permutar los hijos deja fijos Φ (y γ). Por tanto, certificar *todos los planares* de orden $\leq p$ subsume las condiciones de orden abstractas: dos árboles planares con la misma forma abstracta comparten una sola condición.

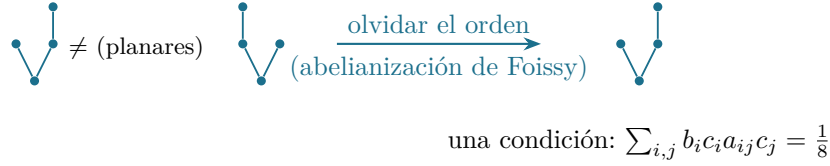


Figura 2: Dos árboles *planares* distintos de orden 4 (hijos en uno u otro orden) van, bajo el morfismo de Hopf olvidadizo de Foissy, a un mismo árbol *abstracto*, y el Lema 1 les da una sola condición de orden. Certificar el catálogo planar certifica por tanto el abstracto.

5. Los certificados: del orden 1 al orden 6

Con la familia hecha finita (§3) y el hueco planar/abstracto salvado (§4), ¿qué podemos certificar de verdad? Todo se reduce ahora a un solo movimiento, repetido: desplegar el motor sobre un árbol y un tableau hasta un objetivo aritmético cerrado, y luego `norm_num`. Elegimos `norm_num` y no `decide` por una razón concreta—sobre $\mathbb{Q}$ el núcleo no puede reducir el `Nat.gcd` dentro de la normalización de `Rat`, así que `decide` se atasca, y `native_decide` añadiría un axioma de confianza en el compilador que rechazamos. *Guarda ese inciso: reaparece en la cima del ascenso.* Con `norm_num` ascendemos: Euler da orden 1, el trapecio explícito orden 2, el RK4 clásico orden 4, y Dormand–Prince `DOPRI5`—el integrador detrás de `ode45` y de `RK45` de SciPy—orden 5, todo sobre $\mathbb{Q}$.

El paso siguiente cambia el *terreno*: Gauss–Legendre de tres etapas es *implícito* y su tableau es *irrational*, vive en $\mathbb{Q}(\sqrt{15})$, donde `norm_num` no tiene de dónde agarrar. Así que lo mudamos: un anillo computable de pares `Q15` con $\sqrt{15}^2 = 15$ incrustado en la multiplicación, de modo que $\sqrt{15}$ nunca aparece como símbolo—cada condición de orden colapsa a dos identidades *racionales*, y el mismo `norm_num` las cierra. Esto lleva el método implícito de coeficientes algebraicos limpiamente hasta el orden 4.

El orden 6 es donde el ascenso se gana su desenlace. La misma ruta de `norm_num`, llevada a las 65 condiciones de orden 6, *se derrumba*—no en el catálogo (que se reduce en $\sim 0,1$ s) sino en la aritmética racional de cada condición, cuyo trabajo simbólico sobre $\mathbb{Q}$ en los árboles grandes agota la memoria (§6). Y aquí se redime aquel inciso del primer párrafo: no podíamos usar `decide` por el `gcd` de $\mathbb{Q}$, así que salimos de $\mathbb{Q}$. Despejar denominadores (escalar por $D = 360$) mueve el tableau al anillo de *enteros* $\mathbb{Z}[\sqrt{15}]$, donde, por homogeneidad del peso elemental, la condición de orden se vuelve la identidad entera pura $\gamma(t) \Phi(D \cdot A, D \cdot b, t) = D^{|t|}$. Sobre los enteros, la aritmética GMP del núcleo la evalúa directamente: `decide`—la táctica a la que habíamos renunciado—cierra ahora las 65 condiciones en ~ 1 s, sin axiomas. El ascenso alcanza el orden 6, y lo hace con el movimiento que habíamos apartado, ahora posible gracias a una representación mejor.

Método	Etapas	Tipo	Coeficientes	Orden certificado
Euler	1	explícito	$\mathbb{Q}$	1
Trapezio explícito (Heun)	2	explícito	$\mathbb{Q}$	2 (y no 3)
RK4 clásico	4	explícito	$\mathbb{Q}$	4 (y no 5)
Dormand–Prince DOPRI5	7	explícito	$\mathbb{Q}$	5
Gauss–Legendre $s=3$	3	implícito	$\mathbb{Q}(\sqrt{15})$	$6^\dagger$

Tabla 3: Certificados de condiciones de orden comprobados por máquina y sin axiomas. El abanico cubre métodos explícitos e implícitos, coeficientes racionales y algebraicos, comprobados por máquina hasta el orden 6. † El orden 6 de Gauss se certifica en Lean por la ruta entera $\mathbb{Z}[\sqrt{15}]$ con `decide` (Sección 6) y se confirma de forma independiente de tres maneras (Python y Sage).

t									
$\gamma(t)$	1	2	3	6	4	8	12	24	5
$\Phi_{\text{RK4}}(t)$	1	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{6}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{12}$	$\frac{1}{24}$	$\frac{5}{24}$
$= 1/\gamma(t)?$	✓	✓	✓	✓	✓	✓	✓	✓	×

Tabla 4: El RK4 clásico, comprobado por máquina: $\Phi = 1/\gamma$ para todo árbol de orden ≤ 4 , y el primer árbol de orden 5 (la escoba, última columna) falla ($\frac{5}{24} \neq \frac{1}{5}$). Así que el RK4 queda certificado de orden 4 y *no* de orden 5. Los valores coinciden exactamente con los oráculos de Sage/Python.

Dos certificados negativos—el RK4 falla una condición de orden 5 ($\Phi = 5/24 \neq 1/5$), Heun falla en el orden 3—fijan el orden *exactamente*, no solo por debajo. Gauss–Legendre $s=3$ queda confirmado de orden 6-y-no-7 por un cálculo independiente en Sage sobre $\mathbb{Q}(\sqrt{15})$ con un algoritmo distinto (suma bruta sobre índices en lugar de la recursión por árboles).

6. Evidencia de verificación

El repositorio compila con `lake env lean (toolchain v4.30.0-rc2)`; todo teorema cabecera—`rk4_order4`, `dp_order5`, `gauss_order4`, `gauss_order6`, el lema de simetrización 1 y los certificados negativos—reporta `#print axioms = [propext, Classical.choice, Quot.sound]`: sin `sorry`, sin axiomas ad hoc, sin `native_decide`. La salida numérica del motor coincide con dos oráculos independientes (`app/bseries.py` sobre $\mathbb{Q}$; un script de Sage sobre $\mathbb{Q}$ y $\mathbb{Q}(\sqrt{15})$).

El orden 6 en Lean: un cambio de fórmula. Gauss–Legendre $s=3$ es *exactamente* de orden 6, confirmado de tres maneras sobre $\mathbb{Q}(\sqrt{15})$ (la recursión por árboles y una suma bruta sobre índices—un algoritmo genuinamente distinto— en Python, y la misma suma bruta en Sage, `validate_order_conditions.sage`): las condiciones se cumplen hasta el orden 6 y fallan las 48 de orden 7, cancelándose el componente $\sqrt{15}$ en cada peso elemental. Certificar esto en Lean exigió un rodeo digno de registrar. La ruta ingenua—el mismo `gauss_check` (`simp` del motor y luego `norm_num` sobre los componentes de $\mathbb{Q}$) que prueba el orden 4—*no escala*: `fin_cases` sobre el catálogo de 65 árboles es barato (se reduce en $\sim 0,1$ s), pero cada una de las 65 llamadas a `norm_num` por condición es costosa, y en los árboles grandes de orden 6 la elaboración agota la memoria (una ejecución llegó

a ~ 20 GB sin terminar). El culpable es la aritmética racional: la normalización con `Nat.gcd` de $\mathbb{Q}$ no se reduce en el núcleo, así que `decide` no está disponible y `norm_num` debe trabajar simbólicamente sobre productos anidados enormes.

El arreglo es cambiar la *representación*, no el método: *despejar los denominadores*. Escalar el tableau por $D = 360$ (el m.c.m. de 36, 9, 15, 30, 24, 18) lo mueve al anillo de *enteros* $\mathbb{Z}[\sqrt{15}]$ (un par de `Int`). Por homogeneidad del peso elemental, $\Phi(D \cdot A, D \cdot b, t) = D^{|t|} \Phi(A, b, t)$, de modo que la condición de orden $\gamma(t) \Phi = 1$ se vuelve la identidad entera

$$\gamma(t) \cdot \Phi(D \cdot A, D \cdot b, t) = D^{|t|} \quad \text{en } \mathbb{Z}[\sqrt{15}].$$

Sobre los enteros esto cierra por `decide`: el núcleo de Lean 4 evalúa la aritmética de `Int` directamente (con respaldo GMP), sin normalización racional y con memoria despreciable. El certificado completo

```
gauss_order6 : satisfiesOrderConditionsInt gaussI_A gaussI_b 360 6
```

—`fin_cases` sobre los 65 árboles planares de `catalog 6`, cada uno descargado por `decide` en el núcleo—compila en ~ 1 s y es **libre de axiomas** (`#print axioms = [propxt, Classical.choice, Quot.sound]`). Hasta donde sabemos, este parece estar entre los primeros certificados de orden *completo* comprobados por máquina de un método de Runge–Kutta implícito con coeficientes algebraicos. El patrón entero-con-`decide` se traslada a cualquier método cuyo tableau se despeje a $\mathbb{Z}[\sqrt{d}]$. (El mismo truco es la vía natural a órdenes aún mayores, donde la enumeración por árbol acaba topando con el crecimiento combinatorio del catálogo—véase la Sección 7.)

Rendimiento. La Tabla 5 da el coste de cada certificado. El fichero entero recompila en ~ 62 s (de los cuales el import es $\sim 1,7$ s); las cifras de abajo son el tiempo de elaboración+núcleo por teorema, aproximado, una sola ejecución en caliente, Lean v4.30.0-rc2 en una CPU de escritorio. Dos lecturas destacan. Primera: el cuello de botella nunca es el catálogo—`fin_cases` sobre los 65 árboles de `catalog 6` cuesta solo ~ 131 ms; el tiempo es la aritmética de cada condición. Segunda, y el sentido del Brick 6: el *mismo* método de Gauss cuesta $\sim 3,5$ s a orden 4 (9 árboles, `norm_num` sobre $\mathbb{Q}(\sqrt{15})$) pero solo $\sim 0,5$ s a orden 6 (65 árboles, `decide` sobre $\mathbb{Z}[\sqrt{15}]$)—siete veces más árboles, un orden de magnitud menos tiempo, puramente por la representación entera. En cambio la ruta `norm_num` intentada a orden 6 sobre $\mathbb{Q}(\sqrt{15})$ no termina, agotando la memoria (una ejecución llegó a ~ 20 GB).

Certificado	Método	orden p	<code>catalog p</code>	anillo / táctica	tiempo
<code>rk4_order4</code>	RK4	4	9	$\mathbb{Q}$ / <code>norm_num</code>	$< 0,4$ s
<code>dp_order5</code>	DOPRI5	5	23	$\mathbb{Q}$ / <code>norm_num</code>	$\sim 1,5$ s
<code>gauss_order4</code>	Gauss $s=3$	4	9	$\mathbb{Q}(\sqrt{15})$ / <code>norm_num</code>	$\sim 3,5$ s
<code>gauss_order6</code>	Gauss $s=3$	6	65	$\mathbb{Z}[\sqrt{15}]$ / <code>decide</code>	$\sim 0,5$ s

Tabla 5: Coste por certificado (aproximado, una ejecución en caliente). La comprobación del catálogo (`fin_cases`) es ~ 131 ms incluso a orden 6; el coste es la aritmética por árbol. El `decide` entero sobre $\mathbb{Z}[\sqrt{15}]$ certifica las 65 condiciones de orden 6 más rápido que `norm_num` sobre $\mathbb{Q}(\sqrt{15})$ certifica las 9 de orden 4, mientras que la ruta `norm_num` a orden 6 no termina (~ 20 GB).

7. Alcance y límites

Enunciamos la frontera con exactitud. **(1)** Certificamos las condiciones de orden *algebraicas* $\gamma(t)\Phi(t) = 1$; *no* formalizamos el teorema analítico de Butcher de que satisfacerlas equivale a un

error local $O(h^{p+1})$ (eso requiere convergencia de B-series, diferenciales elementales y el desarrollo de Taylor del flujo—trabajo futuro). **(2)** Los árboles son planares; el puente a las condiciones abstractas es el Lema 1, no una formalización del morfismo de Hopf completo de Foissy.

(3) Formalización, no matemática nueva. La matemática es clásica en todo momento (Butcher [1]; Foissy [5]). La clave (Teorema 1) es una *reducción* que habilita la formalización—una comprobación de catálogo finito es como procede informalmente una prueba de libro de texto—y su contenido aquí es solo que la reducción, su completitud y los certificados resultantes están comprobados por máquina. La contribución es la implementación reutilizable, no un teorema nuevo; un analista numérico diría con razón que la novedad está en la formalización más que en la matemática, y estamos de acuerdo.

(4) El marco en álgebra de Hopf es contextual. La Sección 9 sitúa las condiciones de orden como *caracteres* del álgebra de Hopf de Connes–Kreimer/Butcher y enlaza con el artículo compañero [17]; esa perspectiva es aquí motivacional. Los certificados de este artículo usan solo los árboles, orden, γ , Φ y la invariancia por permutación del Lema 1: el coproducto, la antípoda, la ley de grupo de Butcher y el corchete pre-Lie *no* se invocan—están formalizados en [17]. Por eso el título nombra árboles enraizados, no la estructura de Hopf completa.

(5) La afirmación “no formalizado antes” es una búsqueda dirigida, no un teorema. Un desarrollo olvidado podría existir y nos alegraría que nos corrigieran. Nuestra afirmación se apoya en la búsqueda de abajo (realizada en junio de 2026); es dirigida, no exhaustiva. En cada sistema buscamos en el índice y el texto completo los términos *Runge–Kutta*, *Butcher*, *rooted tree*, *order condition(s)* y *B-series*.

Sistema	Fuentes consultadas
Lean 4 / Mathlib	repositorio Mathlib, Loogle, Moogole, Zulip <code>leanprover</code>
Isabelle/HOL	Archive of Formal Proofs (índice + texto completo)
Coq/Rocq	índice <code>opam</code> , <code>coq-community</code> , web

El único resultado sobre métodos RK comprobados por máquina fue Boldo *et al.* [8] (Coq, redondeo en coma flotante de Euler/RK2)—una capa ortogonal, no la teoría de orden.

8. Trabajo previo

La teoría de orden: Butcher [1]; Hairer–Lubich–Wanner [2]; Chartier–Hairer–Vilmart [3]. La lectura en álgebra de Hopf: Connes–Kreimer [4]; el álgebra planar (no conmutativa) y su abelianización, Foissy [5, 6]; el álgebra de árboles binarios planares, Loday–Ronco [12]; relaciones operádicas planar/no planar, Chapoton [7]. Una prueba informal recursiva por árboles del teorema de Butcher está en Bornemann [13]. Software: `NodePy` [9], `BSeries.jl` [10], Sofroniou [11]. Métodos formales sobre RK: Boldo *et al.* [8] (redondeo en Coq, ortogonal). Formalizaciones compañeras: el álgebra de Hopf planar de Connes–Kreimer–Foissy en Lean [17] y el idempotente euleriano / error hacia atrás [18], ambas sobre Mathlib [14].

9. La lectura en álgebra de Hopf: las condiciones de orden como caracteres

¿Por qué es este el mismo objeto que la renormalización y el álgebra pre-Lie libre? La teoría de orden de Butcher y el álgebra de Hopf de Connes–Kreimer de árboles enraizados son dos lecturas de una sola estructura [4, 1, 5]. Los árboles de aquí son una base de esa álgebra de Hopf H ; el

producto de bosques es su multiplicación; el coproducto de cortes admisibles—formalizado en el artículo compañero [17]— es dual a la *composición de flujos numéricos*, la ley de grupo del grupo de Butcher. Un método de B-series es un *carácter* de H (un morfismo de álgebras $H \rightarrow \mathbb{R}$, es decir, multiplicativo en los bosques); para un tableau (A, b) ese carácter es exactamente nuestro peso elemental $t \mapsto \Phi(t)(A, b)$. El flujo exacto a tiempo h es él mismo un carácter, $t \mapsto 1/\gamma(t)$. En este lenguaje, toda la teoría de orden es un solo enunciado:

$$\text{un método tiene orden } p \iff \Phi \text{ y } 1/\gamma \text{ coinciden en } H_{\leq p},$$

es decir, el carácter del método y el del flujo exacto coinciden en las piezas graduadas hasta el orden p . La antípoda formalizada en [17] es el método *inverso* (adjunto); su logaritmo—el idempotente euleriano de [18]— es la ecuación modificada, el error hacia atrás de un método. Las tres formalizaciones son así tres caras de una misma álgebra de Hopf: su *estructura* [17], sus *caracteres* (las condiciones de orden, este artículo) y su lado de *Lie/logaritmo* (el error hacia atrás, [18]). Lo que [17] listaba como el *applied payoff* que abría su agenda, este artículo lo entrega.

10. Qué desbloquea esto

El núcleo reutilizable—árboles, orden, γ , Φ , el generador por presupuesto, la clave— es el sustrato de dos pasos siguientes ya a la vista: el *error hacia atrás* certificado (la ecuación modificada, vía el idempotente euleriano del Artículo II) y las leyes de sustitución/composición (la imagen operádica de Chapoton). El comprobador certificado de condiciones de orden es el primer peldaño.

Referencias

- [1] J. C. Butcher, *Coefficients for the study of Runge–Kutta integration processes*, J. Austral. Math. Soc. **3** (1963), 185–201.
- [2] E. Hairer, C. Lubich, G. Wanner, *Geometric Numerical Integration*, Springer, 2006.
- [3] P. Chartier, E. Hairer, G. Vilmart, *Algebraic structures of B-series*, Found. Comput. Math. **10** (2010), 407–427.
- [4] A. Connes, D. Kreimer, *Hopf algebras, renormalization and noncommutative geometry*, Comm. Math. Phys. **199** (1998), 203–242.
- [5] L. Foissy, *Les algèbres de Hopf des arbres enracinés décorés I*, Bull. Sci. Math. **126** (2002), 193–239. (arXiv:math/0105212; álgebra planar/no conmutativa, autodualidad, abelianización Prop. 81.)
- [6] L. Foissy, *Les algèbres de Hopf des arbres enracinés décorés II*, Bull. Sci. Math. **126** (2002), 249–288.
- [7] F. Chapoton, *Operads and combinatorial Hopf algebras of trees* (y trabajos afines sobre estructuras de árboles planares vs. no planares).
- [8] S. Boldo, F. Faissolle, *et al.*, *Formally-verified round-off error analysis of Runge–Kutta methods*, J. Automated Reasoning **68** (2024).
- [9] D. I. Ketcheson, *NodePy*, J. Open Source Software **5**(55):2515, 2020.

- [10] D. I. Ketcheson, H. Ranocha, *Computing with B-series*, ACM TOMS **49**(2), 2023.
- [11] M. Sofroniou, *Symbolic derivation of Runge–Kutta methods*, J. Symbolic Comput. **18** (1994), 265–296.
- [12] J.-L. Loday, M. O. Ronco, *Hopf algebra of the planar binary trees*, Adv. Math. **139** (1998), 293–309.
- [13] F. Bornemann, *Runge–Kutta methods, trees, and Mathematica*, 2002, arXiv:math/0211049. (Una prueba informal recursiva por árboles del teorema de orden de Butcher.)
- [14] The mathlib Community, *The Lean Mathematical Library*, CPP 2020; toolchain Lean 4 v4.30.0-rc2.
- [15] OEIS Foundation, *Sequence A000081: unlabeled rooted trees*, <https://oeis.org/A000081>.
- [16] OEIS Foundation, *Sequence A000108: Catalan numbers (ordered/planar rooted trees)*, <https://oeis.org/A000108>.
- [17] C. Marín, *How a Tree Remembers Its Cuts: the Connes–Kreimer–Foissy Hopf algebra of rooted trees, machine-checked in Lean 4*, 2026 (artículo compañero).
- [18] C. Marín, *The Eulerian Idempotent and backward error of B-series, in Lean 4*, 2026 (artículo compañero).